1 Mutex (Mutual Exclusion)

Definition: class is a synchronization primitive that can be used to protect shared data from being simultaneously accessed by multiple threads.

Libraries Required: <mutex>

1.1 Lock

1.2 Try Lock

1.3 Unlock

2 Lock Guard(s)

Definition: mutex wrapper that provides a convenient RAII-style mechanism for owning a mutex for the duration of a scoped block.

Libraries Required: <mutex>
Different types of lock guards:

1. lock guard
2. scoped lock
3. unique lock
4. shared lock

3 Thread

Definition: A lightweight sub-task or flow of control that lives inside a process.

3.1 Join

3.2 Detach

4 JThread

Definition: a modern replacement for std::thread that represents a single thread of execution. The "j" stands for joining, which directly highlights its core purpose: it implements the RAII (Resource Acquisition Is Initialization) pattern for managing thread lifecycles.

5 Process

Definition: An active instance of a program with its own isolated memory space and system resources

6 Async

Definition: a mechanism the function f asynchronously (potentially in a separate thread which may be part of a thread pool) and returns a std::future that will eventually hold the result of that function call.

6.1 Launch Policy

Launch policies allow the programmer when to trigger the async operation(s).

6.1.1 launch::async

Launches a new thread to call your given **function** as if a thread object

6.1.2 launch::deferred

Your given **function** is deferred until the shared state of the returned future is accessed with wait or get (member functions in a future).

7 Future

Definition: is a class template that acts as a placeholder or a handle for a value (or exception) that will eventually be computed by an asynchronous operation.

Libraries Required: <future>

8 Promise

Definition: is a mechanism to store a value or an exception that will be asynchronously retrieved by another thread.

Libraries Required: <future>

9 Package Task

Definition: class template that wraps any callable target (such as a function, lambda expression, or bind expression) so that its return value or thrown exception can be retrieved asynchronously via a std::future.

Libraries Required: <future>

10 Atomics

Definition: operations or types that guarantee indivisible execution and inter-thread synchronization without the use of explicit locks (like mutexes).

Libraries Required: <atomic>

11 Stop Token

Definition: is a thread-safe, lightweight C++20 class that provides a read-only view of a shared stop-state to check whether an asynchronous operation or a std::jthread has been asked to stop.

Libraries Required: <stop_token>

12 Condition Variables

Definition: a mechanism that allows one or more threads to be suspended (put to sleep) until another thread modifies a shared state and notifies them.

Libraries Required: <condition_variable>